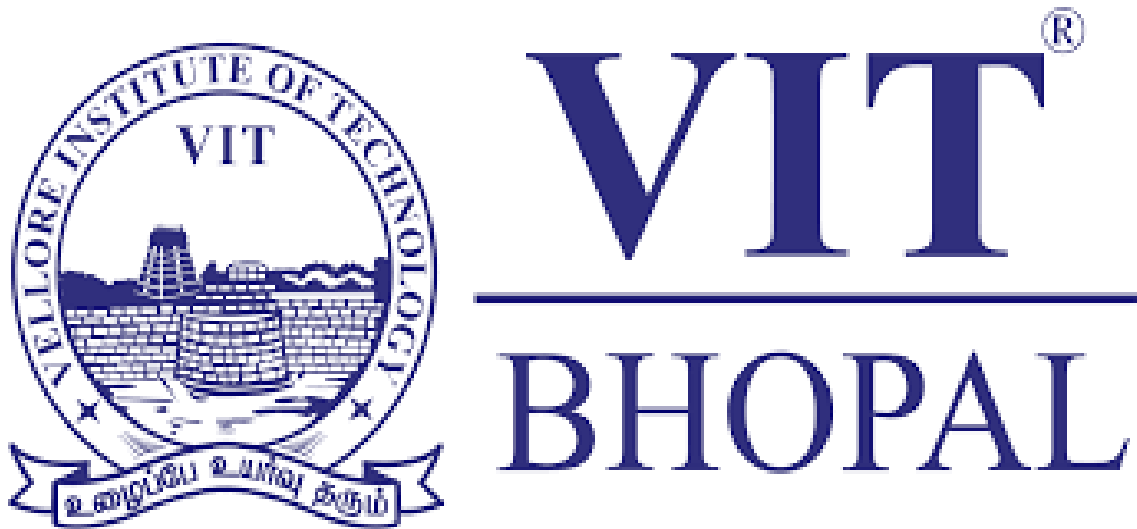




# **LIBRARY MANAGEMENT SYSTEM**

**Submitted by: Aadarsh Batra**

**Registration Number : 25BAI10872**

**Program: B.TECH (AI and ML)**

**Faculty Name : Manorama Chauhan**

**Slot : F11 + F12**

# INTRODUCTION

Libraries, whether in a school, college, or public institution, involve a recurring set of operations: cataloguing books, registering members, tracking who currently holds which book, and enforcing simple rules such as borrowing limits and due dates. Doing this by hand does not scale much past a few dozen books, and even small institutions benefit from a system that can answer questions instantly - "Is this book available?", "How many books does this member currently hold?".

This project implements the model of a Library Management System in Java. Its focus is entirely on designing a correct, well-structured, and testable object model along with the service layer that operates on it. The system covers the core lifecycle of a library book: cataloguing, searching, member registration, issuing, returning and fine calculation for overdue returns, with a full transaction history maintained for reporting.

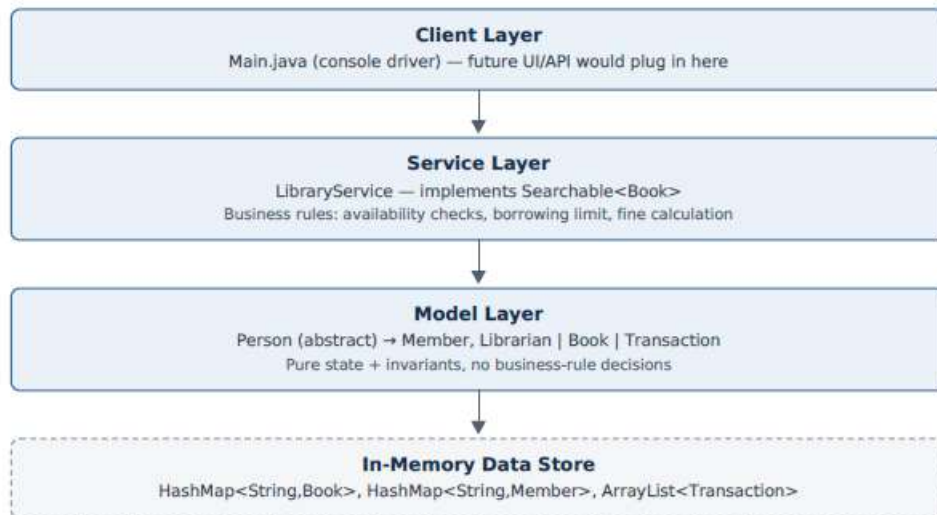
# PROBLEM STATEMENT

Design and implement an object-oriented backend model that represents books, members, librarians, and transactions as first-class entities; enforces business rules (availability checks, borrowing limits, fine calculation) consistently through a single service layer; and is verifiable through automated unit tests rather than manual inspection alone.

## FUNCTIONAL REQUIREMENTS

1. The system shall allow a librarian to add a new book (with ISBN, title, author, genre, and number of copies) to the catalogue.
2. If a book with the same ISBN already exists, adding it again shall increase the existing copy count rather than create a duplicate entry.
3. The system shall allow a librarian to remove a book from the catalogue by ISBN.
4. The system shall allow searching the catalogue by a keyword matched against title or author (case-insensitive).
5. The system shall allow registering a new member with a unique member ID, name, and email.
6. The system shall reject registration of a member ID that is already in use. The system shall allow issuing a book to a member, provided the book exists, has at least one available copy, and the member has not reached the maximum borrowing limit (3 books).
7. The system shall allow returning a previously issued book and shall calculate a fine if the return date is past the due date (14 days from issue).
8. The system shall maintain a chronological transaction log of every issue and return event.

# SYSTEM ARCHITECTURE



Custom checked exceptions (`BookNotFoundException`, `MemberNotFoundException`, `BookNotAvailableException`, `BorrowingLimitExceededException`, `DuplicateMemberException`) cut across the Service and Client layers - they are raised in the Service layer and must be explicitly handled by whatever calls it, which forces every caller to make a conscious decision about failure handling rather than letting errors pass silently.

# DESIGN DIAGRAMS

Two actors interact with the system: the Librarian, who manages the catalogue and membership, and the Member, who searches for and borrows books.

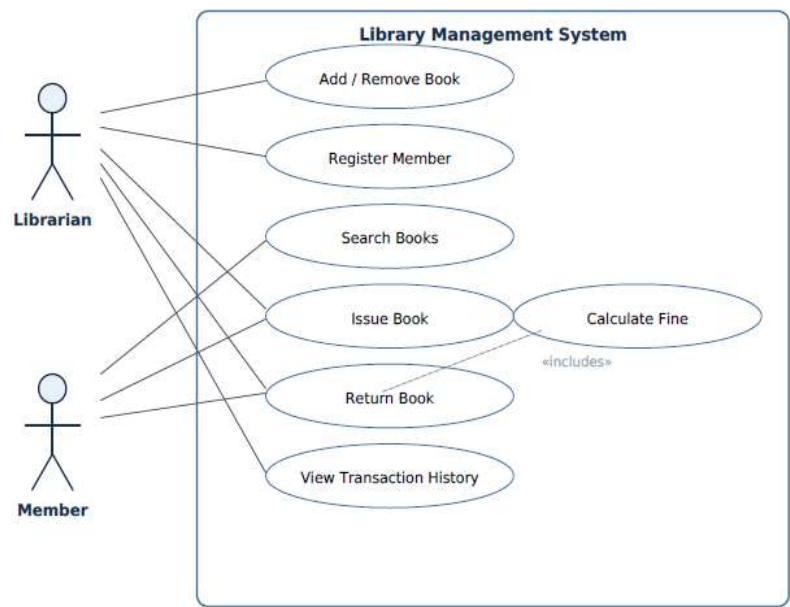
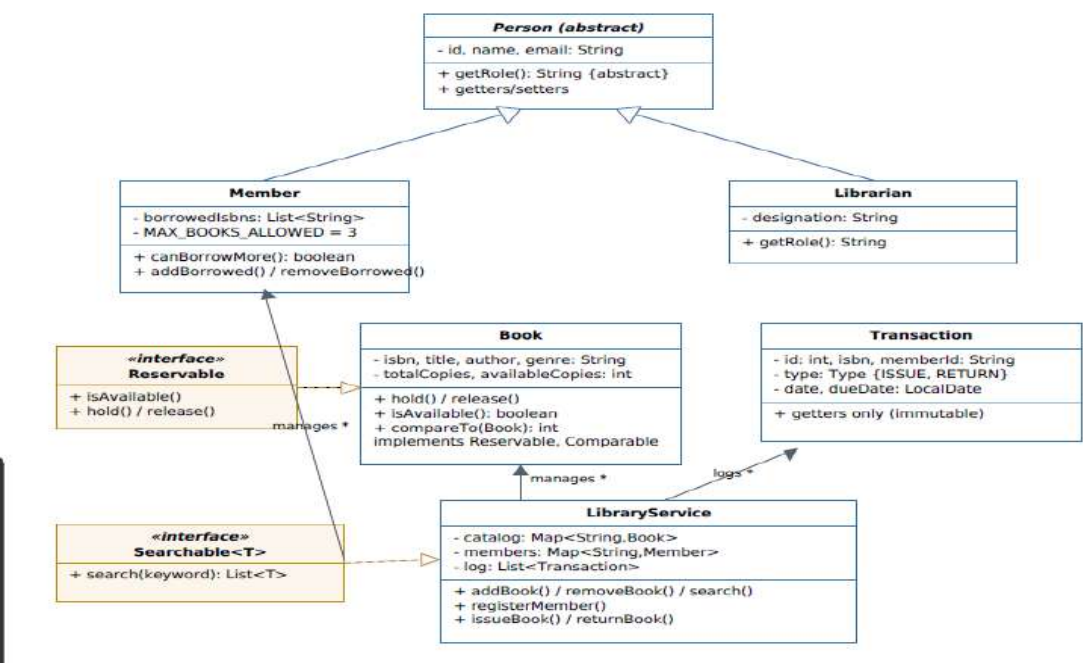


Fig 7.1 — Use Case Diagram

## CLASS DIAGRAM



## WORKFLOW

The diagram below traces the decision flow for the highest-risk operation in the system - issuing a book - since this is where most of the business rules apply at once.

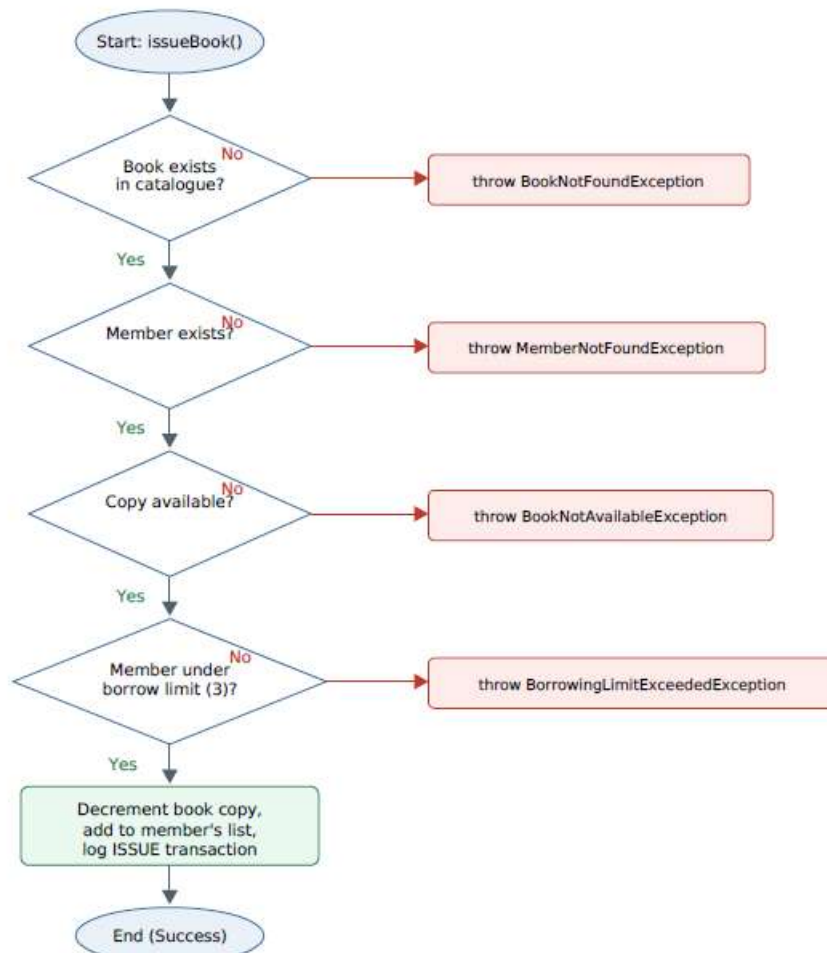


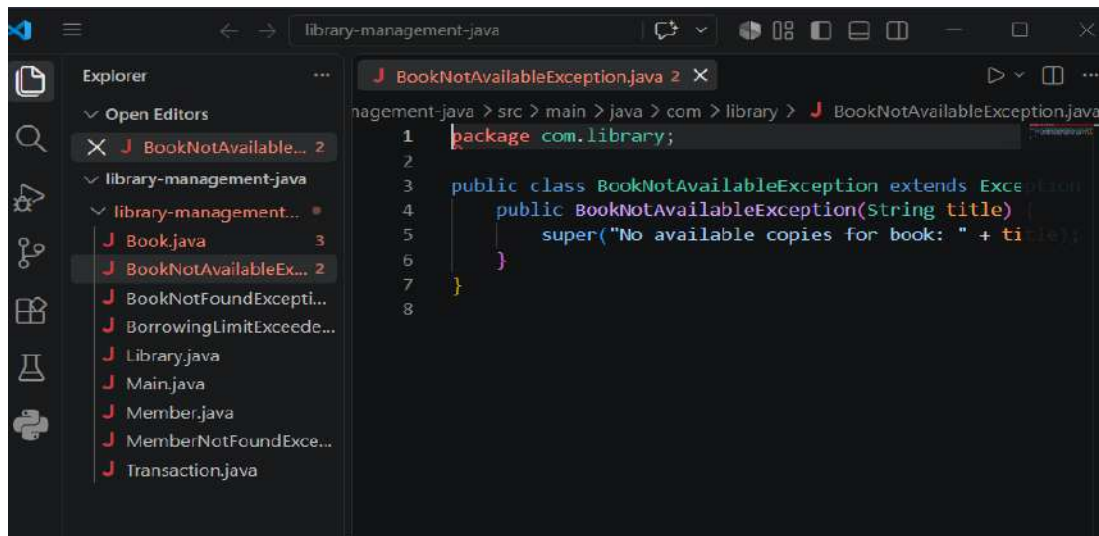
Fig 7.2 — Workflow / Activity Diagram for Issue Book

# DESIGN DECISIONS AND RATIONABLE

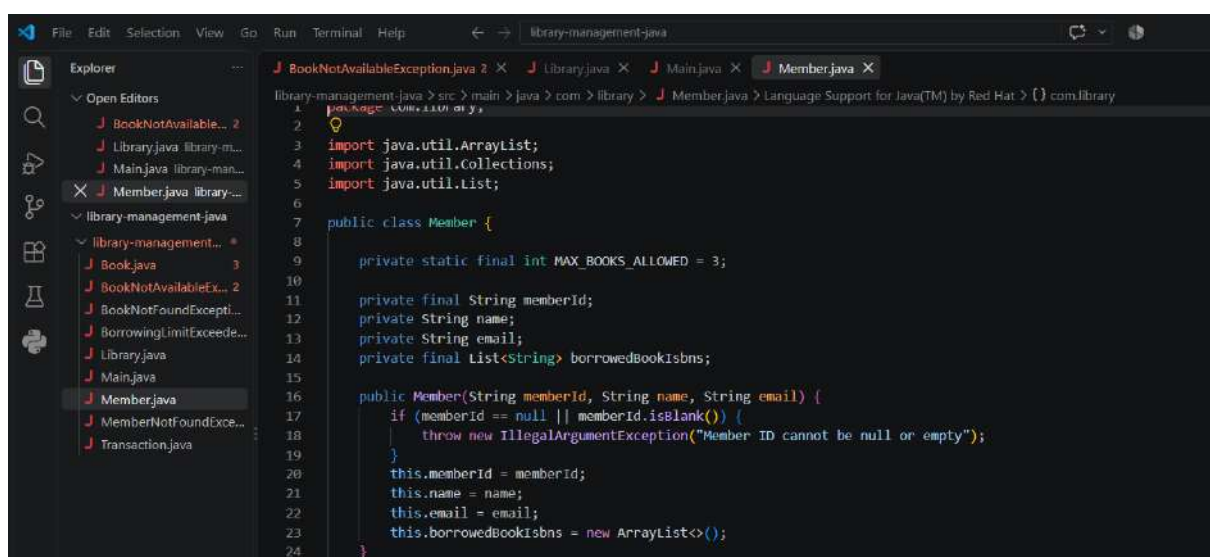
| DECISION                                                                                                | RATIONABLE                                                                                                                                                                                                                                                                                                      |
|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Abstract Person base class for Member and Librarian                                                     | Both roles share identity fields (id, name, email). Pulling this up avoids duplicated fields/getters and demonstrates inheritance meaningfully rather than just for its own sake - <code>getRole()</code> is genuinely different per subclass.                                                                  |
| Reservable interface on Book instead of public setters                                                  | Copy counts should only ever change by exactly one, through <code>hold()/release()</code> . A public <code>setAvailableCopies(int)</code> would let any caller set an inconsistent value; the interface keeps the invariant ( $0 \leq \text{available} \leq \text{total}$ ) enforced inside Book itself.        |
| Checked exceptions rather than return codes / null                                                      | Returning null for "book not found" pushes the null-check burden onto every caller and is easy to forget. A checked exception forces the compiler to demand handling, which is safer for a system where silently ignoring a failed issue/return would corrupt the copy count.                                   |
| Single LibraryService class rather than splitting into BookService / MemberService / TransactionService | At this scale, issuing a book genuinely needs to coordinate <i>both</i> a book and a member atomically (check both, then mutate both). Splitting into separate services would mean passing state back and forth for no real decoupling benefit - it would just move the coordination logic into a fourth class. |
| In-memory storage (HashMap / ArrayList) instead of a database                                           | The assignment scope is the backend object model, not persistence. HashMap gives $O(1)$ lookup by ISBN/member ID which is what the operations need; a real database can be swapped in later behind the same LibraryService API                                                                                  |

# IMPLEMENTATION DETAILS

- Custom exception example
- 



- Coordinating validation



# SCREENSHOTS /RESULTS

Console output from running Main.java, which exercises the full flow: catalogue setup, member registration, issuing, a deliberate failure case (last copy already taken), returning, transaction history, and search.

```
Terminal
PS D:\STUDY\java-n\library-management-java> & 'C:\Program Files\Eclipse Adoptium\jdk-25.0.3-hotspot\bin\java
.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\LOQ\AppData\Roaming\Code\User\workspaceStorage\
0a78d94b3949d497905561da7ba01e38\redhat.java\jdt_ws\library-management-java_4745dced\bin' 'com.library.Main'
=== Initial Catalog ===
Book[ISBN=978-0596009205, Title='Head First Design Patterns', Author='Freeman & Robson', Genre='Programming', A
vailable=1/1]
Book[ISBN=978-0134685991, Title='Effective Java', Author='Joshua Bloch', Genre='Programming', Available=3/3]
Book[ISBN=978-0132350884, Title='Clean Code', Author='Robert C. Martin', Genre='Programming', Available=2/2]

=== Issuing Books ===
Issued: Txn[#1001, ISSUE, ISBN=978-0134685991, Member=M001, Date=2026-09-18, Due=2026-10-02]
Issued: Txn[#1002, ISSUE, ISBN=978-0596009205, Member=M002, Date=2026-09-18, Due=2026-10-02]
Expected failure: No available copies for book: Head First Design Patterns

=== Catalog After Issuing ===
Book[ISBN=978-0596009205, Title='Head First Design Patterns', Author='Freeman & Robson', Genre='Programming', A
vailable=0/1]
Book[ISBN=978-0134685991, Title='Effective Java', Author='Joshua Bloch', Genre='Programming', Available=2/3]
Book[ISBN=978-0132350884, Title='Clean Code', Author='Robert C. Martin', Genre='Programming', Available=2/2]

=== Returning a Book ===
Book returned. Fine due: 0.0

=== Transaction History ===
Txn[#1001, ISSUE, ISBN=978-0134685991, Member=M001, Date=2026-09-18, Due=2026-10-02]
Txn[#1002, ISSUE, ISBN=978-0596009205, Member=M002, Date=2026-09-18, Due=2026-10-02]
Txn[#1003, RETURN, ISBN=978-0134685991, Member=M001, Date=2026-09-18]

=== Search ===
Search 'clean': [Book[ISBN=978-0132350884, Title='Clean Code', Author='Robert C. Martin', Genre='Programming',
Available=2/2]]
Search author 'martin': [Book[ISBN=978-0132350884, Title='Clean Code', Author='Robert C. Martin', Genre='Progra
mming', Available=2/2]]
PS D:\STUDY\java-n\library-management-java>
```

# TESTING APPROACH

The test suite below was executed against the real `com.library.model` and `com.library.service` classes (via a JUnit-5-compatible reflection runner, since this report-generation environment has no outbound access to Maven Central to pull the full JUnit engine). On a normal development machine, `mvn test` runs the exact same test sources through the full JUnit Platform and produces an equivalent Surefire report.

```
● ● ● Terminal — running unit tests

com.library.model.BookTest
  PASS  newBookStartsWithAllCopiesAvailable
  PASS  holdReducesAvailableCopiesByOne
  PASS  holdingLastCopyMakesItUnavailable
  PASS  holdingWithNoCopiesLeftThrows
  PASS  releaseGivesACopyBack
  PASS  releasingMoreThanTotalThrows
  PASS  constructorRejectsBlankIsbn
  PASS  constructorRejectsNegativeCopies
  PASS  addCopiesIncreasesBothTotalAndAvailable
  PASS  booksAreEqualIfIsbnMatchesEvenWithDifferentTitles

com.library.service.LibraryServiceTest
  PASS  issuingReducesAvailableCopies
  PASS  issuingUnknownIsbnThrows
  PASS  issuingToUnknownMemberThrows
  PASS  issuingLastCopyToTwoMembersThrowsOnSecond
  PASS  memberCannotExceedBorrowingLimit
  PASS  returningGivesTheCopyBack
  PASS  returningOnTimeMeansNoFine
  PASS  registeringDuplicateMemberIdThrows
  PASS  searchMatchesTitleCaseInsensitively
  PASS  searchMatchesAuthorToo
  PASS  searchWithNoMatchesReturnsEmptyList
  PASS  transactionLogRecordsIssueAndReturn
  PASS  addingMoreCopiesOfExistingBookMergesInsteadOfDuplicating

.....
Tests run: 23, Passed: 23, Failed: 0
```

# CHALLENGES FACED

**Deciding where validation belongs.** Early on it was tempting to let Book itself decide whether an issue is allowed.

**Ordering of precondition checks in `issueBook()`.** Checking availability before the borrowing limit (or vice versa) changes which exception a caller sees first when both conditions fail. Settled on: existence checks first, then availability, then the member's limit so the error a caller sees always matches the first thing they could realistically fix.

**Calculating fines without storing extra state.** Rather than adding a "fine" field to Book or Member, the due date already lives on the ISSUE transaction reusing it for the fine calculation avoided introducing a second source of truth for the same date.

**Testing without a running JUnit engine in this environment.** Verifying the tests actually pass (not just compile) required

# Learnings & Key Takeaways

1. Encapsulation is most valuable exactly where invariants matter - Book's copy counters are the clearest example: no external code can ever push them out of range.
2. Checked exceptions, used sparingly and for genuinely recoverable conditions, communicate intent better than a generic Runtime Exception or a magic return value.
3. A generic interface is worth introducing even for a single implementer, if it clearly names a capability the system has and might reuse.
4. Writing the unit tests alongside the service logic (not after) surfaced the borrowing-limit boundary case and the "issue same last copy twice" case before they became bugs.

# Future Enhancements

1. Persist data with a real database (e.g. via JDBC or Spring Data) following the ER model .
2. Add a console menu or REST API on top of `LibraryService` .
3. Reservation queue for books with zero copies available.
4. Configurable loan period / fine rate per genre or member type.

## REFERENCES

1. Oracle. *The Java® Tutorials*. <https://docs.oracle.com/javase/tutorial/>
2. Oracle. *Java SE 17 API Documentation*.  
<https://docs.oracle.com/en/java/javase/17/docs/api/>
3. Apache Maven Project. *Maven - Introduction to the POM*.  
<https://maven.apache.org/guides/introduction/introduction-to-the-pom.html>
4. JUnit Team. *JUnit 5 User Guide*.  
<https://junit.org/junit5/docs/current/user-guide/>
5. Bloch, J. *Effective Java* (3rd Edition). Addison-Wesley, 2018. (Referenced for encapsulation and exception-design conventions used in this project.)